# Raytrace: two focused improvements

The selected sphere_shadow kernel renders the same image in 530.8805 ms/frame versus 795.61825 ms for upstream: 33.27% less runtime, or 1.4987x speedup. It combines scalar sphere intersection with shadow-ray reuse only. The earlier full bundle was faster, but is not the selected final implementation.

The benchmark renders a deterministic 100 x 100 scene containing seven spheres, a checkerboard halfspace and two point lights. For every primary ray it finds the nearest object, shades the hit, tests light visibility, and follows reflection rays for up to three additional bounces. math supplies scalar functions; array stores RGB bytes; pyperf supplies timing support. No external numerical library replaces the renderer.

| Data structure | Role |
|---|---|
| Vector and Point | Three coordinate attributes holding Python float objects. Operators create temporary geometric objects. |
| Ray | Origin plus normalized direction; construction invokes vector arithmetic and normalization. |
| Scene and surfaces | Ordered object/material pairs, point lights and recursive shading state. |
| Canvas | array of RGB bytes with unchanged clamping, conversion and pixel order. |

The reference algorithm performs 179,457 sphere queries per frame. There are only a few objects, so our chosen approach keeps traversal and geometry intact and reduces the Python work required for each query. A fixed count is a property of this workload and traversal, not proof that every possible algorithm must use that count.

Final source: variants/bm_raytrace_upstream_opt.py, kernel=sphere_shadow. Latest raw session: results/session_raytrace_20260929-193534_678/. Reports follow the six sections required by Project.pdf.

# Connect the profile to the source

The course VM runs release CPython 3.10.12 for clean timing. Each arm has 11 fresh processes, identical work per process and GC disabled. perf stat, native perf record with python3-dbg, cProfile and the official pyperformance baseline execute separately. Only the unprofiled timer decides the 7% runtime target. Configuration details remain in the run manifests rather than dominating this report. Raytrace uses four frames per timed process; cProfile and native sampling use one frame.

The retained upstream sources are byte-identical to the pyperformance 1.14.0 wheel, independently checked during packaging. Baseline wrappers call upstream benchmark functions directly; optimized wrappers temporarily replace selected methods/functions and then call the same upstream workload. This is a custom comparison harness around genuine pyperformance code. The official suite run is a separate upstream-only cross-check, not a claim of optimized-suite measurement.

| Baseline observation | Mechanism to investigate |
|---|---|
| 98,172 Ray constructions; 109,887 normalizations | The shadow loop rebuilds the same normalized ray for each object tested. |
| 509,873 dot calls; 277,865 Point subtractions | Sphere intersections repeatedly construct a displacement Vector and dispatch small methods. |
| 452,955 Vector constructions | Many short-lived geometric intermediates, not a large scene data set. |
| 318 / 1,068 native self samples in _PyEval_EvalFrameDefault | Bytecode evaluation and operand/reference handling are prominent; this is not solely function-call overhead. |

The bottleneck hypothesis is excessive interpreted work around small numeric expressions. The evidence does not establish a universal instruction cost per call or a precise percentage of release time spent doing useful floating-point work. The strong causal test is whether a source transformation removes the predicted calls and instructions while preserving the workload and improving clean runtime.

Source evidence: upstream/bm_raytrace_upstream.py, raw/cprofile_baseline.pstats and .txt, perf/report_baseline_self.txt, flame/baseline_top30_stacks.txt in the selected baseline directory.

# Reuse the shadow ray

For a fixed surface point p and light l, the direction l-p is invariant while testing candidate occluders. Upstream constructs Ray(p, l-p) inside that loop. Ray.__init__ normalizes the direction, so repeating construction also repeats subtraction, magnitude, square root, scaling and temporary allocation.

```
# Upstream structure
for obj, surface in scene.objects:
    t = obj.intersectionTime(Ray(p, l - p))
    if t is not None and t > EPSILON:
        return False

# Selected structure for ordinary benchmark objects
ray = Ray(p, l - p)
for obj, surface in scene.objects:
    t = obj.intersectionTime(ray)
    if t is not None and t > EPSILON:
        return False
```

The empty-scene case returns before constructing a ray. Object order, the strict positive shadow threshold and first-blocker early exit remain unchanged. This is reuse within one visibility query, not caching answers across rays or altering which objects can cast shadows.

The implementation applies this path to stock scenes and objects; custom primitives or materials retain upstream visibility behavior, so a custom primitive that mutates a ray cannot affect later tests through reuse. This check has a runtime cost and remains inside the measured workload.

| Count per frame | Upstream | sphere_shadow |
|---|---|---|
| Light visibility queries | 10,666 | 10,666 |
| Ray constructions | 98,172 | 26,000 |
| Normalizations | 109,887 | 37,715 |

The measured reduction is exactly 72,172 Ray constructions and normalizations. Removing their displacement and scale temporaries accounts for 144,344 fewer Vector objects. The unchanged visibility-query count confirms that less work is done per query rather than fewer light tests.

# Scalar sphere arithmetic and exactness

Upstream forms cp = sphere.centre - ray.point, then calls cp.dot(direction) and cp.dot(cp). The scalar method computes cp_x, cp_y and cp_z directly and spells out the same expressions. It removes the temporary Vector and the two dot dispatches per sphere query while preserving binary64 operation order.

```
cp_x = centre.x - ray.point.x
cp_y = centre.y - ray.point.y
cp_z = centre.z - ray.point.z
v = (cp_x * d.x) + (cp_y * d.y) + (cp_z * d.z)
q = (cp_x * cp_x) + (cp_y * cp_y) + (cp_z * cp_z)
disc = (radius * radius) - (q - v * v)
return None if disc < 0 else v - math.sqrt(disc)
```

Stock-type checks retain the original method for custom Point, Ray, Vector or Sphere behavior. No approximate square root, fused multiply-add, reciprocal rewrite, altered hit epsilon or precision reduction is introduced. Locals still hold boxed Python floats; the optimization eliminates geometric objects and dispatch, not all numeric allocation.

Only Sphere.intersectionTime and Scene._lightIsVisible are patched. Scoped installation restores the original methods after success or exception. The upstream scene, camera, nearest-hit helper, guard methods, checkerboard behavior and pixel conversion stay active.

The saved VM gate compares every frame in a four-frame 100 x 100 batch and restores baseline behavior. Pixels are bit-identical; the 100 x 100 SHA-256 prefix is 11f2c36b02044f11. Additional tests compare primitive float bits for randomized and boundary cases, raw colours before quantization, varied scenes, custom-object fallback and method restoration. These are explicit tested contracts, not a proof for arbitrary monkey-patched Python objects.

Considered but not selected: guard specialization, camera reuse, direct nearest-hit scanning and checkerboard cleanup remain available in full; slots changes object storage semantics. Some exploratory isolated changes were neutral/slower, but the measured full bundle succeeded. Unconditional guard deletion, approximate geometry and fixing upstream rendering quirks were rejected because they change behavior.

# Latest timing: 33.27% less runtime

| Per work unit | Upstream | Selected optimized |
|---|---|---|
| Median (ms) | 795.618 | 530.880 |
| Mean (ms) | 796.127 | 530.012 |
| Sample SD (ms) | 6.389 | 2.645 |
| Minimum (ms) | 787.061 | 524.328 |
| Maximum (ms) | 807.551 | 533.336 |
| Coefficient of variation | 0.803% | 0.499% |




Unprofiled process medians, normalized by the identical batch size in both arms. Profiler elapsed times are excluded.

The median saving is 264.73775 ms/frame. Runtime reduction is 100 x (1 - optimized/baseline), while speedup is baseline/optimized. The 49.87% throughput gain is a different metric and is not substituted for the course runtime criterion.

A 20,000-resample independent-arm bootstrap gives a 95% percentile interval of 32.81%-33.91% runtime reduction. This describes the observed process variability. The official upstream pyperformance cross-check has 60 values and reports 801 +/- 7 ms, consistent in scale with the custom baseline but collected using its own harness.

The arms ran in sequential blocks, not interleaved A/B pairs. Bootstrap intervals resample the 11 processes independently and describe sampling variability only; they do not cover shared-host drift. PMU events were multiplexed and some VM events were unsupported. These limitations do not turn the large observed timing difference into a precise hardware-stall diagnosis.

# Where the saved work went

| Event per work unit | Upstream (M) | Optimized (M) | Change |
|---|---|---|---|
| instructions | 5055.13 | 3280.53 | -35.11% |
| cycles | 1924.39 | 1283.46 | -33.31% |
| L1-dcache-loads | 1642.92 | 1049.20 | -36.14% |
| branches | 648.77 | 419.11 | -35.40% |

| cProfile count / frame | Upstream | sphere_shadow |
|---|---|---|
| Sphere queries | 179,457 | 179,457 |
| Vector constructions | 452,955 | 129,154 |
| dot calls | 509,873 | 78,787 |
| Point subtractions | 277,865 | 26,236 |
| All recorded calls | 3,092,298 | 1,171,138 |

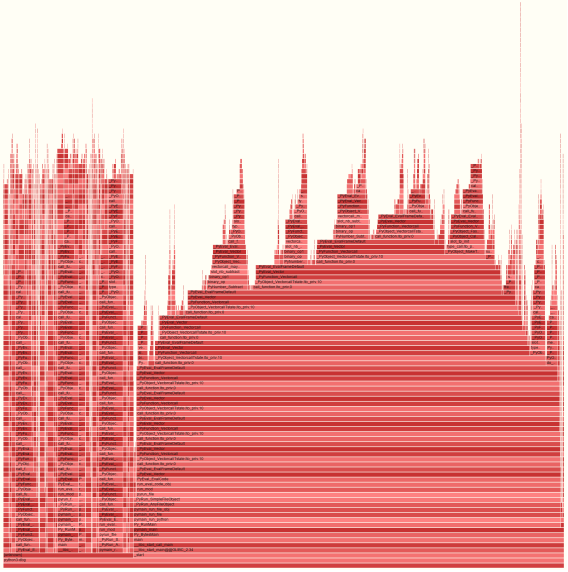
Vector construction falls by 323,801 = 144,344 from shadow reuse + 179,457 from scalar sphere intersections. dot calls fall by 431,086 = 72,172 avoided normalizations + 2 x 179,457 sphere dot calls. Point subtraction falls by 251,629 = 72,172 shadow displacements + 179,457 sphere displacements. These exact count identities link the two changes to the observed profile.

Instructions fall 35.11% and cycles 33.31%, close to the 33.27% runtime improvement. IPC changes from 2.627 to 2.556: the program wins by executing less work, not by improving IPC. L1 load count falls 36.14% and load misses fall 25.60%, yet the miss rate rises from 2.70% to 3.15% because the denominator shrinks faster. This is not evidence of a cache-driven speedup.

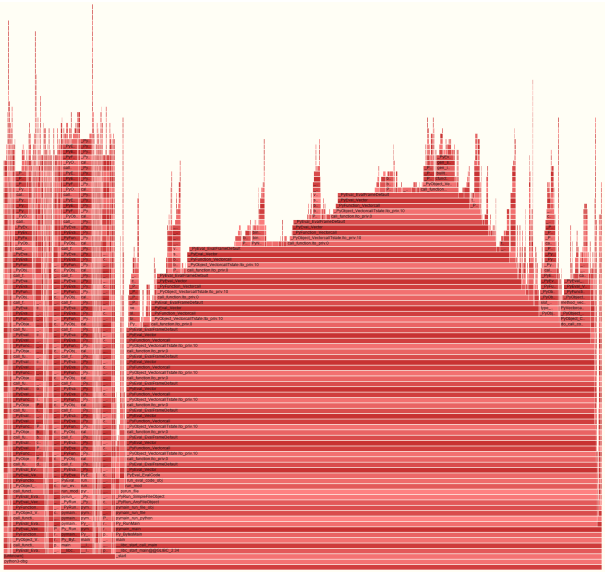
Counter table uses perf/stat_*.txt means consistently, divided by four. The .csv files are a separate counter invocation. Negative top-down fractions and unsupported LLC events are excluded from interpretation; event multiplexing is retained in the raw data.

# Why the flame graphs still look similar

Upstream: 1,068 samples



sphere_shadow: 648 samples



Overview of the original native flame graphs. Each graph fills its own width; equal widths do not mean equal runtime. Open the retained SVGs for searchable, zoomable symbol labels.

A flame graph groups sampled call stacks. Width is inclusive sample weight, height is stack depth, and horizontal position is not time. Self samples count only the leaf; inclusive parents overlap. The native profile contains CPython implementation frames, while cProfile supplies Python function names and exact call counts. Together they connect source changes to removed work.

| Native debug self category | Baseline | Optimized |
|---|---|---|
| Evaluator | 318 (29.78%) | 179 (27.62%) |
| Broad dictionary/attribute helpers | 157 | 85 |
| Frame management | 86 | 38 |
| All samples | 1,068 | 648 |

Both kernels still use the same recursive renderer and CPython interpreter. Repeating evaluation/vectorcall blocks therefore remain. Weighted mean native depth changes only from 68.1 to 66.8, even while the recorded sample count and Python calls fall. A graph normalized to fill the page can look similar after substantial absolute work is removed.

These are debug-build sampled mechanisms, not release-time Amdahl fractions. Broad dictionary groups include more than lookups. Unknown stack frames remain in the raw data. The recorded command uses a 5-million-cycle period; a legacy SVG caption saying F=999 is not the actual sampling setting. Original SVGs and perf dumps are retained unmodified.

# Comparison with the earlier full bundle

| Session / kernel | Paired baseline | Optimized | Reduction |
|---|---|---|---|
| 20 Sep / full | 791.215 ms | 452.696 ms | 42.78% |
| 29 Sep / sphere_shadow | 795.618 ms | 530.881 ms | 33.27% |

The selected two-change kernel is not faster than full. Its optimized median is 17.27% higher across the two sessions. The paired baseline is only 0.56% higher; comparing optimized/baseline ratios gives a 16.62% relative regression. This normalization is informative but cannot remove every between-session effect or identify each component's individual contribution.

| Work per frame | sphere_shadow | Historical full |
|---|---|---|
| Vector constructions | 129,154 | 105,497 |
| All recorded calls | 1,171,138 | 881,684 |
| Sphere tests | 179,457 | 179,457 |
| Instructions | 3,280.5 M | 2,843.4 M |

The new selection deliberately keeps camera, guard, nearest-hit and checkerboard paths in upstream form. The 23,657 extra Vector constructions equal 19,800 camera temporaries plus 3,857 checkerboard temporaries that full removed. The new shadow path also checks stock-scene eligibility on each query, whereas full can reuse eligibility while its patched render method is active. Both factors help explain the extra calls; the data is not a clean per-feature ablation.

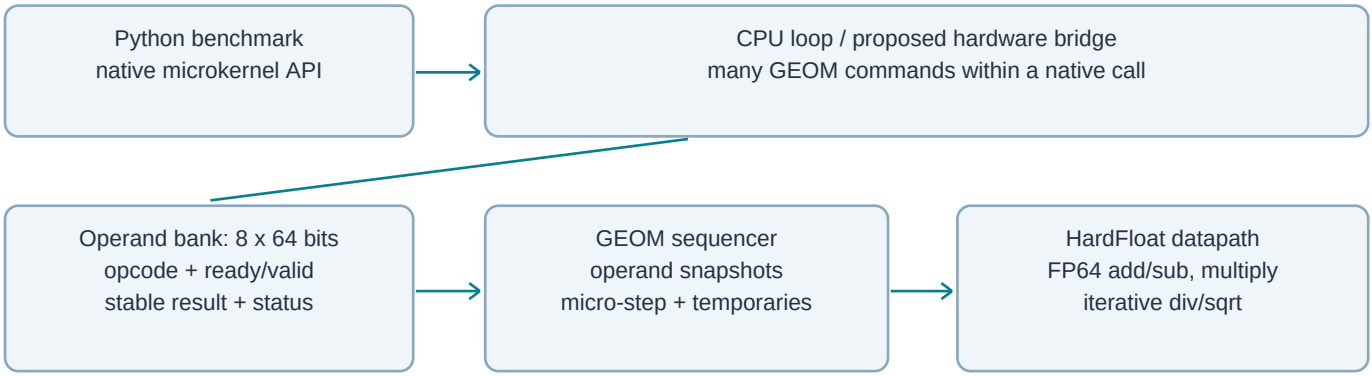
We retain sphere_shadow because the requested final scope is exactly these two optimizations, with a simpler explanation and a large margin above 7%. full remains a valid, faster historical alternative rather than a failed idea. Only the latest raw raytrace pair is packaged; historical full statistics and the comparison calculation are retained in analysis/latest_results_analysis.json.

A direct balanced same-session sphere_shadow-versus-full experiment would be needed to estimate the implementation difference more tightly. No such new VM run is claimed in this submission.

# An ordered sphere-intersection command

The hardware proposal targets the remaining scalar sphere test after the selected sphere_shadow software improvements. The query count is unchanged: 179,457 sphere tests per frame, with 6,934 nonnegative discriminants. A SPHERE command keeps intermediate numbers in hardware registers and sequences the complete ordered expression. This reduces CPU/device command count compared with submitting individual arithmetic operations, while early rejection skips square root for the other 96.14% of queries.

The CPU supplies cp = center – origin, normalized ray direction and radius. Hardware computes v = (cp.x*dx + cp.y*dy) + cp.z*dz; q = (cp.x*cp.x + cp.y*cp.y) + cp.z*cp.z; disc = r*r – (q – v*v); then returns v – sqrt(disc), or ROOT_NONE for a finite negative discriminant. CPU traversal retains nearest-hit t > –1e–5, shadow t > +1e–5, tie order, halfspace handling and early exits. Expression-level agreement does not replace full-image validation of a future integrated adapter.




Solid datapath boxes describe included RTL; CPU-to-device binding is proposed integration.
Ordered operations round separately. One outstanding command; no FMA or DMA engine.

Architecture and interface. The submitted GEOM unit contains eight 64-bit operand registers, ten 64-bit temporaries, a micro-operation controller, HardFloat add/subtract and multiply logic, and one iterative divide/square-root core. Each operation rounds separately to binary64, round-to-nearest ties-to-even; gradual underflow is retained and FMA is absent. A 50 MHz target is specified, without claiming timing closure. The controller waits for actual arithmetic completion, rather than using behavioral real arithmetic with artificial delays.

Included RTL, interface specification, source provenance, validation and analytical model: hardware/. Three annotated interactive diagrams are in hardware/diagrams/; they are conceptual, not cycle-accurate simulations.

# Interface, software stack and validation

| Interface | Contract |
|---|---|
| Operands | op_we + op_addr[2:0] + op_wdata[63:0], idle writes |
| Command | cmd_opcode[7:0], cmd_valid / cmd_ready |
| Response | rsp_result[63:0], rsp_status[3:0], rsp_valid / rsp_ready |
| Control | clk, active-low rst_n; one command outstanding |
| Status | OK, ROOT_NONE, NONFINITE, BADOP |

The host finishes writes before issuing a command. Writes while busy are ignored; a response remains stable until consumed. Reset aborts in-flight work. The CPU retains traversal, original error behavior, and a fallback for exceptional results. The proposed stack is benchmark → native adapter → processor bridge → GEOM. A CPU-native numerical control is included, but no deployed processor bridge or custom instruction is claimed. Moving a loop into C and accelerating it in hardware are separate contributions.

Trade-offs and validation. Shared arithmetic and 1,152 bits of operand/temporary storage limit replicated logic; the single outstanding command limits overlap. Add/multiply critical paths may require pipelining, which changes the cycle estimate. No area, power or frequency result is claimed without synthesis and technology data. Saved checks pass 20,000 ordered dot expressions, 20,000 sphere expressions, exceptional-status cases, and RTL semantic elaboration. No RTL simulation or physical experiment was performed. The supplied animations are included as conceptual aids; the authoritative submitted architecture is hardware/diagrams/architecture.svg.

Area and power trade-offs: sharing arithmetic limits duplicated logic and register storage, but serialization can extend active time. Holding unused operands stable reduces switching; a clock-enable or gated implementation could save idle power, at an implementation cost. More lanes or deeper pipelining could raise throughput while adding registers, clock load and routing. Lower instantaneous power does not guarantee lower energy per frame. These are design trade-offs, not numerical power or area results.

Interactive: hierarchical block diagram

Interactive: command and datapath flow

Interactive: complete workload stream

Included RTL, interface specification, source provenance, validation and analytical model: hardware/. Three annotated interactive diagrams are in hardware/diagrams/; they are conceptual, not cycle-accurate simulations.

# A conditional estimate, with explicit costs

Performance estimate. At two controller cycles per simple operation, 57 assumed core cycles per iterative operation plus two wrapper edges, one cycle per operand write and command/response cycles, a no-root query costs 37 cycles and a root query 98. The 57-cycle input is a conservative planning assumption pending simulation; the controller itself uses completion handshakes. The latest query mix gives 141.258 ms of device work at 50 MHz. An illustrative extra 0.10 microseconds of packing/bridge work per command adds 17.946 ms, for 159.203 ms total offload cost.

Against the current 530.8805 ms/frame software runtime, $T_{est} = 530.8805(1 - f) + 159.20336$ ms. Here f is the fraction actually removed by the complete offload boundary, not a debug-profile percentage. At f = 0.30 the estimate is approximately break-even; f = 0.40 gives 477.732 ms (10.01% reduction), and f = 0.50 gives 424.644 ms (20.01%). At f = 0.20 it is 9.99% slower. These are conditional predictions, not achieved hardware results. Equivalent CPU-native and hardware-backed runs are required to isolate a hardware benefit.

The earlier packet-based design informed the choice of a smaller command interface. The retained proposal uses actual HardFloat logic and an explicit transaction budget. The reported software result remains the measured sphere_shadow improvement; additional hardware speedup remains a conditional design estimate.

| Assumed removable fraction f | Estimated time | Runtime change |
|---|---|---|
| 20% | 583.908 ms | 9.99% slower |
| 30% | 530.820 ms | Approximately break-even |
| 40% | 477.732 ms | 10.01% reduction |
| 50% | 424.644 ms | 20.01% reduction |

The unresolved measurement is the release-build fraction removed by an integrated adapter. Debug-profile shares and cProfile timings do not supply it. The batch boundary, native CPU control and complete device transactions must all be timed before reporting an achieved hardware improvement.

Included RTL, interface specification, source provenance, validation and analytical model: hardware/. Three annotated interactive diagrams are in hardware/diagrams/; they are conceptual, not cycle-accurate simulations.

# What the final submission establishes

sphere_shadow removes redundant shadow-ray construction and sphere-specific vector temporaries while retaining the original traversal and arithmetic. The latest matched pair gives 33.27% less runtime with identical images. full remains faster in the historical comparison, but the final selected scope is the two requested changes. Call counts, instructions and native samples all support the same removed-work explanation.

The shared GEOM proposal supplies an explicit hardware datapath, controller and numerical interface. Its estimated benefit is separated from measured software gains, and native CPU execution is separated from a proposed device bridge. The hardware documents state validation coverage and area/power/frequency assumptions; no measured hardware speedup or achieved clock is claimed.

```
./script_raytrace.sh --variant both --loops 4
LOOPS=4 ./tools/ab_timing.sh raytrace 16
```

| Deliverable | Location |
|---|---|
| Required report and runner | report_raytrace.txt; script_raytrace.sh |
| PDF and reviewed sources | docs/reports/report_raytrace.pdf<br>docs/reports/raytrace_source.json |
| Selected raw evidence | results/latest_raytrace_baseline<br>results/latest_raytrace_optimized |
| Comparison and analysis | results/latest_comparison_raytrace<br>analysis/latest_results_analysis.json |
| HDL, software interface and diagrams | hardware/docs/HARDWARE_DESIGN.md<br>hardware/diagrams/ |
| Prompt record and requirements mapping | prompt.txt<br>docs/SUBMISSION_CHECKLIST.md |

Sources: user-supplied report drafts and hardware notes (hashes in docs/provenance/source_inputs.json); the selected raw runs; byte-verified pyperformance 1.14.0 sources; CPython-native perf profiles. This report supersedes the supplied draft's default and unsupported hardware claims. Software measurements remain original VM evidence.